Lab 8 Report: TinyML Motion Classification and Tiny Ensemble Learning

Part 1

Edge Impulse public project link: https://studio.edgeimpulse.com/public/1006736/live

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 34 ms, inference 563 us, anomaly 0 ms, postprocessing 67 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 30 ms, inference 579 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
```

Figure 1. Deployed Edge Impulse model identifying Lift and Idle states.

Part 2

The notebook was run through the required cells and the optional recorded-IMU validation cell was skipped because no imu_500_rows.csv file was provided.

Notebook Results Summary

Model / artifact	Result	Notes
Input windows	Train: 21350 x 600; Test: 8479 x 600	100 time steps x 6 IMU features.
Raw branch classifier	0.98 test accuracy	Encoder + classifier branch before compression.
Standard-scaled branch classifier	0.99 test accuracy	Best single branch before compression.
Min-max branch classifier	0.98 test accuracy	Third ensemble branch.
Stacked ensemble	1.00 test accuracy	Meta-classifier on concatenated branch outputs.
Compressed int8 raw branch	44.55 KB; 0.9671 TFLite accuracy	Pruned + QAT + int8.
Compressed int8 standard branch	44.55 KB; 0.9518 TFLite accuracy	Pruned + QAT + int8.
Compressed int8 min-max branch	44.61 KB; 0.9499 TFLite accuracy	Pruned + QAT + int8.
Compressed int8 meta-classifier	3.63 KB; 0.9908 TFLite accuracy	Final stacked decision model.

Question 1: Model Architecture and Ensemble Flow

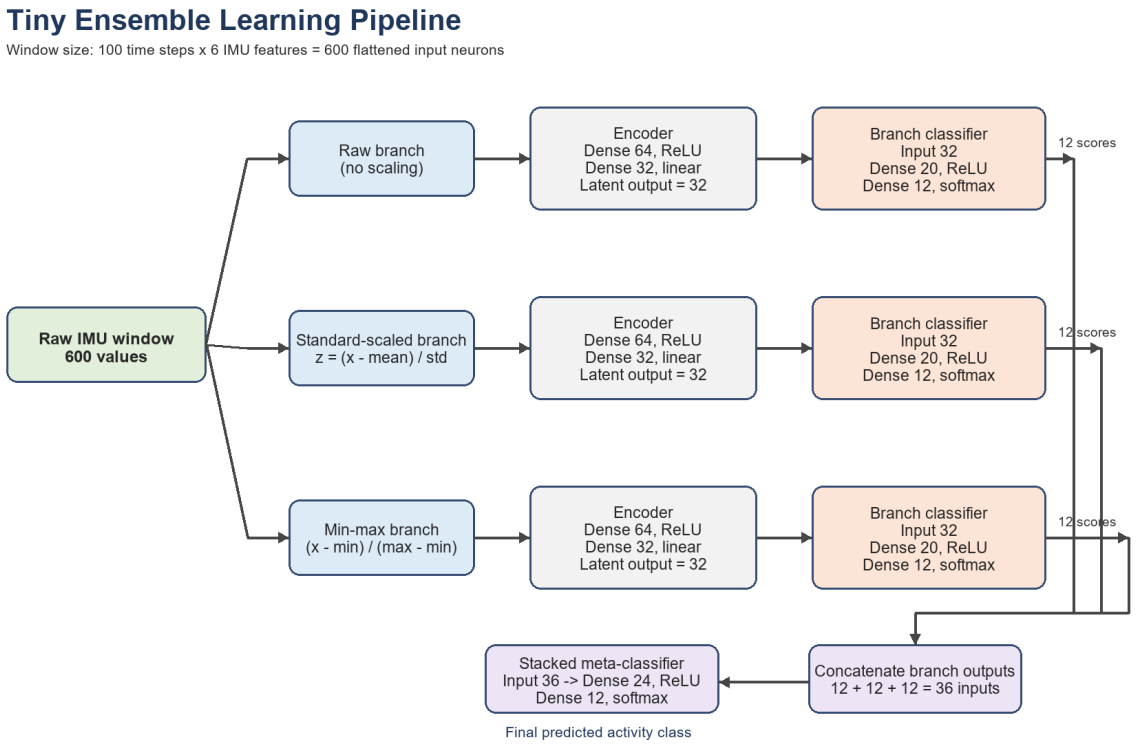

Figure 2. Full three-branch stacked ensemble architecture used in the notebook.

Each input window contains 100 time steps with 6 IMU features, so the flattened input dimension is 600. The ensemble uses three parallel input representations: raw sensor values, standard-scaled sensor values, and min-max-scaled sensor values.

Each branch uses the same encoder architecture: Dense(64, ReLU) followed by Dense(32, linear). The 32-neuron latent vector is then passed to a branch classifier with Dense(20, ReLU) and Dense(12, softmax). Each branch therefore outputs 12 class scores.

The three 12-value branch outputs are concatenated into a 36-neuron stacked input. The meta-classifier architecture is Dense(24, ReLU) followed by Dense(12, softmax), producing the final activity prediction. During training there are three autoencoders, three branch classifiers, and one stacked meta-classifier. During Arduino deployment, the encoder and classifier are flattened into three branch TFLite models, plus one meta-classifier TFLite model.

Question 2: Pruning and QAT Methodology

The notebook compresses the branch pipelines and the stacked meta-classifier using a pruning-then-QAT workflow.

- Pruning is applied with tensorflow_model_optimization.sparsity.keras.prune_low_magnitude using a PolynomialDecay schedule from 10% initial sparsity to 80% final sparsity. The schedule begins at step 0 and ends at step 500, and each model is pruned for 5 epochs.
- After pruning, strip_pruning removes the pruning wrappers and leaves a normal Keras model with the learned zero-valued weights. This is important because QAT expects a standard Keras model structure and the pruning wrappers are training-only objects.
- QAT is applied with quantize_model, then each quantized model is trained for 5 epochs. This lets the model adapt to fake-quantized weights and activations before final int8 export.

- The notebook also extracts masks from the stripped pruned model and enforces those masks during and after QAT. This prevents QAT training from changing previously pruned zero weights back into nonzero values, preserving the intended sparsity.
- The final TFLite converter uses tf.lite.Optimize.DEFAULT, a representative dataset of up to 200 training samples, TFLITE_BUILTINS_INT8 operations, and int8 input/output tensors. Representative data is needed so TensorFlow Lite can calibrate activation ranges and choose quantization scales and zero-points.
- Pruning reduces the number of useful parameters and can reduce computation when sparsity is exploited. Int8 quantization reduces model storage and tensor memory from 32-bit floats to 8-bit integers, which is important for the limited flash and RAM on the Arduino Nano 33 BLE Sense.

Question 3: Arduino Deployment and Real-World Behavior

```
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0781, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0078, 0.9141, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.5195, 0.0000, 0.0000, 0.0000, 0.4766, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000
Min-max-scaled model scores: 0.0039, 0.0000, 0.3945, 0.0000, 0.1484, 0.0117, 0.0742, 0.3242, 0.0000, 0.0000, 0.0000, 0.0430
Meta-classifier scores: 0.0039, 0.2148, 0.0430, 0.0039, 0.0156, 0.3555, 0.0195, 0.0039, 0.0078, 0.0352, 0.2930, 0.0000
----------------------------------------
Final prediction: Waist bends forward
Class index: 5
Confidence score: 0.3555
----------------------------------------
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.4883, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.4883, 0.0195, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.1211, 0.0000, 0.0000, 0.0000, 0.0000, 0.8750, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.5391, 0.0000, 0.0898, 0.0000, 0.0000, 0.2969, 0.0664, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0000, 0.0078, 0.1094, 0.0000, 0.0039, 0.0156, 0.0039, 0.8086, 0.0000, 0.0352, 0.0117, 0.0000
----------------------------------------
Final prediction: Knees bending (crouching)
Class index: 7
Confidence score: 0.8086
----------------------------------------
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.4102, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.4102, 0.1758, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.0234, 0.0000, 0.0000, 0.0000, 0.2734, 0.2734, 0.0000, 0.4297, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.5977, 0.0000, 0.0898, 0.0000, 0.0000, 0.2969, 0.0039, 0.0000, 0.0000, 0.0078
Meta-classifier scores: 0.0000, 0.0742, 0.1406, 0.0078, 0.0117, 0.0781, 0.0234, 0.0195, 0.0078, 0.3242, 0.3047, 0.0039
----------------------------------------
Final prediction: Jogging
Class index: 9
Confidence score: 0.3242
----------------------------------------
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.9922, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039, 0.0000, 0.0039
Min-max-scaled model scores: 0.0000, 0.0000, 0.8867, 0.0000, 0.0195, 0.0000, 0.0000, 0.0664, 0.0000, 0.0000, 0.0000, 0.0234
Meta-classifier scores: 0.0000, 0.0000, 0.9727, 0.0000, 0.0039, 0.0078, 0.0000, 0.0000, 0.0039, 0.0000, 0.0078, 0.0039
----------------------------------------
Final prediction: Lying down
Class index: 2
Confidence score: 0.9727
----------------------------------------
```

Figure 3. Serial Monitor output from the deployed ensemble model.

Window	Final prediction	Confidence
1	Lying down, class index 2	0.9727
2	Lying down, class index 2	0.9336
3	Lying down, class index 2	0.9258

For the resting board condition, the repeated final label was Lying down with high confidence. This is acceptable as a stable-board output because the mHealth training labels describe human activities rather

than Arduino board states; a flat, motionless board has a steady gravity vector and low gyroscope activity, which most closely resembles a stationary posture such as Lying down.

The result is not a perfect semantic match, since the board was not actually attached to a person who was lying down. The main issues are the mismatch between the original mHealth sensor placement and the Arduino board placement, possible axis-orientation differences, live sensor noise, quantization effects, and feature-scaling sensitivity. In the screenshot, earlier windows also showed less stable labels such as Waist bends forward, Knees bending, and Jogging, which indicates that the live board data does not always match the training distribution cleanly.

Two practical improvements would be to collect Arduino-specific training data for the exact deployment setup and to add a temporal smoothing rule such as majority voting over several windows. Additional improvements would include axis calibration, checking unit conversion against the training dataset, and using a confidence threshold to reject uncertain predictions.

Question 4: Deployment Behavior on the Arduino

Yes. The deployment showed unexpected behavior: the model often predicted the same class, especially Lying down, even when the board condition was only a flat resting Arduino rather than a human activity. In other windows, the output jumped among unrelated classes before returning to Lying down.

This can happen because the model was trained on the mHealth dataset, where the IMU was placed on a human subject and the labels represent whole-body activities. The Arduino test uses a different physical device, sensor placement, board orientation, and motion pattern. As a result, the live accelerometer and gyroscope distributions may be shifted from the training data even when the scaling constants are applied correctly.

Possible causes include axis orientation mismatch, differences in acceleration and gyroscope units, calibration offsets, sensor noise, quantization error, and the fact that a short 2-second flattened window may be dominated by gravity direction rather than meaningful activity dynamics. The stacked meta-classifier can also amplify the strongest branch opinion, so repeated high scores for class 2 can make the final prediction stay on Lying down.